

Register Number: 192425207

Name: Anand Paul. D

CO4– AT1

MLA0201 – Fundamentals of Machine Learning

Question 1: Bayesian Network for Disease Prediction

CODE:

```
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
```

```
# Create Bayesian Network
```

```
model = DiscreteBayesianNetwork([
    ('Obesity', 'Diabetes'),
    ('HighSugar', 'Diabetes')
])
```

```
# CPT for Obesity
```

```
cpd_obesity = TabularCPD(
    variable='Obesity',
    variable_card=2,
    values=[[0.7], [0.3]]
)
```

```
# CPT for High Blood Sugar
```

```
cpd_sugar = TabularCPD(
    variable='HighSugar',
    variable_card=2,
    values=[[0.6], [0.4]]
)
```

```

# CPT for Diabetes
cpd_diabetes = TabularCPD(
    variable='Diabetes',
    variable_card=2,
    values=[
        [0.99, 0.80, 0.70, 0.20],
        [0.01, 0.20, 0.30, 0.80]
    ],
    evidence=['Obesity', 'HighSugar'],
    evidence_card=[2, 2]
)

# Add CPTs
model.add_cpds(cpd_obesity, cpd_sugar, cpd_diabetes)

# Check model
print("Model valid:", model.check_model())

# Perform inference
inference = VariableElimination(model)

result = inference.query(
    variables=['Diabetes'],
    evidence={'Obesity': 1, 'HighSugar': 1}
)

print("\nProbability of Diabetes:")
print(result)

```

OUTPUT:

```
*** Model valid: True

Probability of Diabetes:
+-----+-----+
| Diabetes | phi(Diabetes) |
+-----+-----+
| Diabetes(0) |      0.2000 |
+-----+-----+
| Diabetes(1) |      0.8000 |
+-----+-----+
```

Question 2: Hidden Markov Model for Weather Prediction

```
from hmmlearn import hmm
```

```
import numpy as np
```

```
# Hidden states
```

```
states = ['Sunny', 'Cloudy', 'Rainy']
```

```
# Observations
```

```
observations = ['Dry', 'Wet']
```

```
# Create HMM model
```

```
model = hmm.CategoricalHMM(n_components=3, random_state=42)
```

```
# Initial probabilities
```

```
model.startprob_ = np.array([
```

```
    0.5, # Sunny
```

```
    0.3, # Cloudy
```

```
    0.2 # Rainy
```

```
])
```

```
# Transition probabilities
```

```
model.transmat_ = np.array([
```

```
    [0.6, 0.3, 0.1],
```

```
    [0.2, 0.5, 0.3],
```

```

    [0.1, 0.3, 0.6]
])

# Emission probabilities
model.emissionprob_ = np.array([
    [0.8, 0.2], # Sunny -> Dry/Wet
    [0.5, 0.5], # Cloudy -> Dry/Wet
    [0.2, 0.8] # Rainy -> Dry/Wet
])

# Observation sequence
obs = np.array([[0], [1], [1], [0], [1]])

# Predict hidden states
log_probability, hidden_states = model.decode(obs, algorithm="viterbi")

print("Predicted hidden weather states:")

for state in hidden_states:
    print(states[state])

```

OUTPUT:

```

... Predicted hidden weather states:
    Cloudy
    Rainy
    Rainy
    Rainy
    Rainy

```

Question 3: Markov Random Field for Image Denoising

```
import numpy as np
```

```
# Small noisy grayscale image
```

```
image = np.array([
    [50, 55, 200],
    [52, 60, 210],
    [48, 58, 205]
])
```

```
print("Original Image:")
```

```
print(image)
```

```
# Create updated image
```

```
updated = image.copy()
```

```
# Smooth middle pixels using neighbors
```

```
for i in range(1, image.shape[0] - 1):
```

```
    for j in range(1, image.shape[1] - 1):
```

```
        neighbors = [
            image[i-1][j], # Top
            image[i+1][j], # Bottom
            image[i][j-1], # Left
            image[i][j+1]  # Right
        ]
```

```
        updated[i][j] = int(np.mean(neighbors))
```

```
print("\nUpdated Image:")
print(updated)
```

OUTPUT:

```
... Original Image:
[[ 50  55 200]
 [ 52  60 210]
 [ 48  58 205]]

Updated Image:
[[ 50  55 200]
 [ 52  93 210]
 [ 48  58 205]]
```

Question 4: Conditional Random Field for Named Entity Recognition

```
import sklearn_crfsuite
```

```
# Training sentences
```

```
train_sentences = [
    ["John", "ordered", "Laptop", "with", "order", "ID", "ORD123"],
    ["Rahul", "bought", "Phone", "order", "ORD456"],
    ["Anand", "purchased", "Headphones", "order", "ORD789"]
]
```

```
# Training labels
```

```
train_labels = [
    ["NAME", "O", "PRODUCT", "O", "O", "O", "ORDER"],
    ["NAME", "O", "PRODUCT", "O", "ORDER"],
    ["NAME", "O", "PRODUCT", "O", "ORDER"]
]
```

```
# Function to create word features
```

```
def word_features(sentence, i):
    word = sentence[i]
```

```
features = {  
    'word': word,  
    'lower': word.lower(),  
    'first_letter': word[0],  
    'is_upper': word.isupper(),  
    'is_digit': word.isdigit()  
}
```

```
if i > 0:  
    features['previous_word'] = sentence[i - 1]
```

```
if i < len(sentence) - 1:  
    features['next_word'] = sentence[i + 1]
```

```
return features
```

```
# Convert sentences into features
```

```
X_train = []
```

```
for sentence in train_sentences:
```

```
    features = []
```

```
    for i in range(len(sentence)):
```

```
        features.append(word_features(sentence, i))
```

```
X_train.append(features)
```

```

# Train CRF
crf = sklearn_crfsuite.CRF(
    algorithm='lbfgs',
    max_iterations=100
)

crf.fit(X_train, train_labels)

# New sentence
test_sentence = [
    "Priya", "ordered", "Laptop", "with", "order", "ORD999"
]

# Create test features
X_test = [[
    word_features(test_sentence, i)
    for i in range(len(test_sentence))
]]

# Prediction
prediction = crf.predict(X_test)
print("Predicted labels:")
for word, label in zip(test_sentence, prediction[0]):
    print(word, "->", label)

```

OUTPUT:

```

... Predicted labels:
Priya -> NAME
ordered -> 0
Laptop -> PRODUCT
with -> 0
order -> 0
ORD999 -> ORDER

```



```
5. import pandas as pd

from pgmpy.estimators import HillClimbSearch
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.inference import VariableElimination
```

```
# Create dataset
```

```
data = pd.DataFrame({
    'Age': [
        'Young', 'Young', 'Young', 'Middle',
        'Middle', 'Middle', 'Old', 'Old',
        'Old', 'Young'
    ],
```

```
    'Income': [
        'Low', 'High', 'Low', 'High',
        'High', 'Low', 'High', 'Low',
        'High', 'High'
    ],
```

```
    'Vehicle': [
        'Car', 'Bike', 'Car', 'Car',
        'Bike', 'Bike', 'Car', 'Bike',
        'Car', 'Bike'
    ],
```

```
    'Claim': [
        'No', 'No', 'Yes', 'No',
        'Yes', 'Yes', 'No', 'Yes',
        'No', 'Yes'
    ]
```

```

    ]
})

# Display dataset
print("Customer Dataset:")
print(data)

# Learn Bayesian Network structure
hc = HillClimbSearch(data)

best_model = hc.estimate()

print("\nLearned Network Structure:")
print(list(best_model.edges()))

# Create Bayesian Network
model = DiscreteBayesianNetwork(best_model.edges())

# Learn probabilities from data
model.fit(data)

# Check model
print("\nModel is valid:", model.check_model())

# Perform inference
inference = VariableElimination(model)

result = inference.query(
    variables=['Claim'],

```

```

evidence={
    'Age': 'Young',
    'Income': 'High',
    'Vehicle': 'Bike'
}

)

print("\nProbability of Insurance Claim:")
print(result)

```

OUTPUT:

```

*** Customer Dataset:
   Age Income Vehicle Claim
0  Young   Low    Car    No
1  Young   High   Bike    No
2  Young   Low    Car     Yes
3  Middle  High    Car    No
4  Middle  High   Bike    Yes
5  Middle  Low    Bike    Yes
6    Old   High    Car    No
7    Old   Low    Bike    Yes
8    Old   High    Car    No
9  Young   High   Bike    Yes

```